Feature Engineering and Dimensionality Reduction

Feature Engineering

Feature Engineering is the process of **transforming raw data into meaningful features** that improve model performance.

Better features → Better predictions → Less overfitting.

Why Feature Engineering is important?

- Improves accuracy
- Reduces noise
- Helps algorithms understand patterns
- Reduces training time
- Improves generalization

Types of Feature Engineering

1. Feature Creation

Adding new features from existing data.

Examples:

- Creating **BMI** from weight and height
- Creating **interaction terms** (e.g., $x1 \times x2$)
- Converting date to **day, month, year, weekday**

2. Feature Transformation

Used to make data more suitable for algorithms.

Common transformations:

- **Scaling** (Standardization, Min–Max scaling)
- **Normalization**
- **Log transformation**

3. Handling Missing Values

- Mean/median imputation (for numeric)
- Mode imputation (for categorical)
- KNN imputation

4. Handling Categorical Variables

- One-Hot Encoding
- Label Encoding
- Target Encoding

5. Handling Outliers

- Capping
- Removing extreme values
- Using log scale

6. Feature Selection

Selecting only the most relevant features:

- Filter methods (Correlation, Chi-square)
- Wrapper methods (Forward/Backward selection)
- Embedded methods (Lasso, Ridge)

Relation with Major Algorithms

Linear Regression

- Requires **scaled features**
- Sensitive to outliers
- Feature selection improves R^2
- Interaction features improve modeling of relationships

Logistic Regression

- Needs scaled numeric features
- Categorical encoding essential

KNN

- Highly sensitive to **feature scaling** (distance-based algorithm)
- Irrelevant features degrade accuracy
- Dimensionality reduction improves speed

K-Means

- Also distance-based → must scale features
- One-hot encoding increases dimensions → use PCA
- Removing noisy features improves clustering

Dimensionality Reduction

Dimensionality Reduction means **reducing the number of input features** while preserving the important information.

Why Dimensionality Reduction?

- Reduces computational cost
- Minimizes noise
- Prevents overfitting
- Improves visualization (2D or 3D)
- Helpful for KNN & K-Means (distance-based algorithms)

Types of Dimensionality Reduction Techniques

1. Principal Component Analysis (PCA)
2. Linear Discriminant Analysis (LDA)
3. t-SNE
4. Autoencoders (Deep Learning)

Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a **dimensionality reduction** technique that transforms high-dimensional data into a smaller set of **new features (principal components)** while preserving the **maximum variance** in the data.

It is **unsupervised**, meaning it does not use class labels.

Key points:

- Unsupervised
- Requires feature scaling
- Transformations are linear
- Used in K-Means, KNN for speed and accuracy

Why do we need PCA?

When datasets have many features (dimensions):

- Models become slow
- Data becomes noisy
- Visualization becomes difficult

PCA helps by:

- Removing noise
- Reducing dimensions
- Improving model performance
- Making visualization easier (2D or 3D plots)

How PCA Works (in simple steps)

1. **Standardize the data**

   Scale all features so they have mean 0 and standard deviation 1.

2. **Calculate the covariance matrix**

   It shows how features vary together.

3. **Find eigenvalues and eigenvectors**

   1. **Eigenvalues** → show how much variance each direction captures
   2. **Eigenvectors** → give the direction (Principal Components)

4. **Sort principal components**

   First component captures the *most* variance, second the next, and so on.

5. **Project the data onto selected components**

   This gives new transformed features.

Example

Suppose we have data of **students** measured on three features:

Student	Height (cm)	Weight (kg)	Arm Length (cm)
A	160	55	60
B	170	65	65
C	180	75	70

Notice:

- All three features are *correlated* (as height increases → weight and arm length increase).

Goal of PCA

Reduce **3 features → 2 principal components**, while keeping most of the information (variance).

Q1. A dataset has 3 standardized features X_1, X_2, X_3. The covariance matrix has eigenvalues $\lambda_1 = 5.0,\ \lambda_2 = 1.2,\ \lambda_3 = 0.8$.

a) Which principal component explains the most variance?

b) What proportion of total variance is explained by the first principal component?

Given: eigenvalues $\lambda_1 = 5.0,\ \lambda_2 = 1.2,\ \lambda_3 = 0.8$.

a) Which PC explains the most variance?

→ **PC1** (because λ_1 is largest).

b) Proportion of total variance explained by PC1:

Total variance = $5.0 + 1.2 + 0.8 = 7.0$.

Proportion = $5.0/7.0 = 0.7142857 \approx 71.43\%$.

Q2. A PCA on 6 features gives eigenvalues [4.0, 1.5,0.8, 0.5, 0.1, 0.1].

a) How many components would you keep to explain at least 90% variance?

b) Explain why components with very small eigenvalues can be discarded.

Given: eigenvalues [4.0, 1.5,0.8, 0.5, 0.1, 0.1]. Total = 7.0.

a) Cumulative proportions:

•After PC1: $4.0/7 = 57.14\%$

•After PC1+PC2: $5.5/7 = 78.57\%$

•After PC1+PC2+PC3: $6.3/7 = 90.00\%$

To explain **at least 90%, keep 3 components**.

b) Why discard very small eigenvalues?

Because they contribute negligible variance (information) and often represent noise — removing them simplifies the model, speeds computation, and can reduce overfitting.

Q3 — (Given covariance matrix → compute eigenvals)

Covariance matrix:

$$\Sigma = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

a) Compute the eigenvalues.

b) Which PC explains more variance and what proportion does it explain?

Given: $\Sigma = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$

a) Eigenvalues: solve $|\ \Sigma - \lambda I\ | = 0$:

$$\lambda^2 - 4\lambda + 3 = 0 \Rightarrow \lambda = 3,\ 1$$

b) PC with $\lambda = 3$explains more. Proportion = $3/(3+1) = 3/4 = 0.75 = 75\%$.

Q4 — (Compute proportion — numeric)

Eigenvalues of a covariance matrix are [3, 2, 1].

Calculate the fraction of total variance explained by:

a) PC1, b) PC1+PC2.

Given: eigenvalues [3,2,1] .

$$\text{Total} = 3 + 2 + 1 = 6.$$

a) PC1 proportion = $3/6 = 0.5 = 50\%$.

b) PC1+PC2 proportion = $(3 + 2)/6 = 5/6 \approx 83.33\%$.

Real-Life Example: Face Recognition

An image has thousands of pixels → thousands of features.

PCA reduces these into the most important components called **Eigenfaces**.

This makes face recognition faster and more efficient.